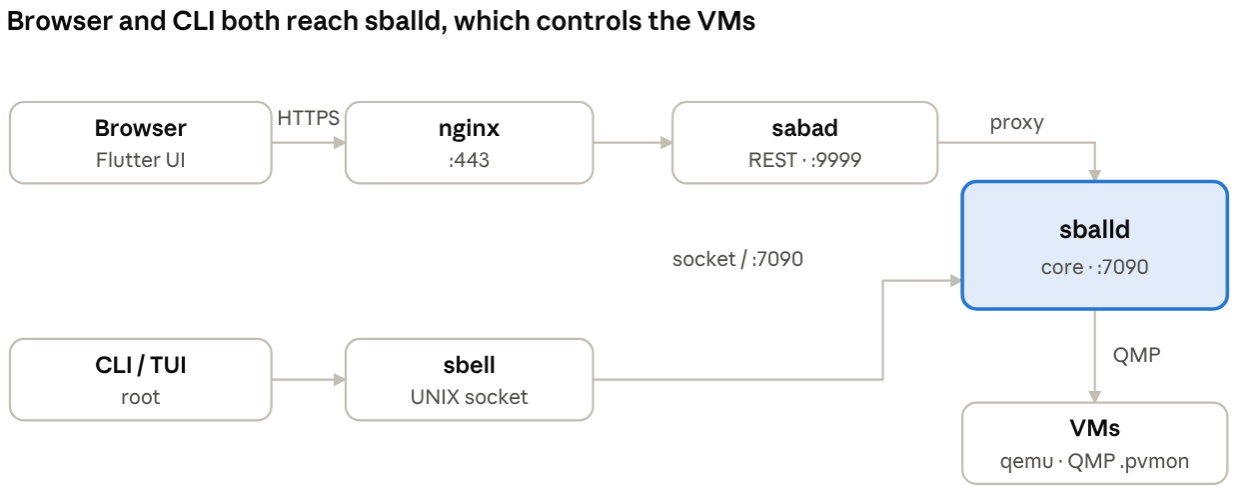
Plaivid CLI & sbell Reference

Oct 27, 2026 · @shaiya

A practical reference for driving Plaivid / PVM from the command line via `sbell` (and the equivalent `sabad` HTTP API), as groundwork for a CLI TUI. Host: `sponge` (spongebob), PVM 7.1.

Overview & architecture

Plaivid is a Flutter web UI backed by the PVM stack on host `sponge` (spongebob, PVM 7.1). Everything the UI does is a command sent to `sballd`, the core daemon; the CLI reaches that same core through `sbell`.



Plaivid request path · browser via nginx/sabad, CLI via sbell, both into sballd

The browser talks HTTPS to nginx, which serves the static UI and proxies API calls to the `sabad` HTTP service; `sabad` forwards to `sballd`. `sbell` skips all of that and speaks to sballd's local UNIX socket directly. Both paths converge on `sballd`, which drives the qemu VMs (including their QMP `.pvmon` control sockets). So anything the web UI can do, `sbell` can do too - that is what makes a CLI TUI possible.

Authentication & sessions

Every action call needs a session token. You get one from `user/login`, pass it to each later call, and drop it with `user/logout`.

```
# login -> token is returned inside <description>
sbell user login "<params><username>manage</username>
<password>SECRET</password></params>"
#   -> <response><status>success</status><action_id>...</action_id>
<description>eyJ...JWT...</description></response>

# use the token (trailing argument) on every subsequent call
sbell vm stop "<vm_key>UUID</vm_key>" "$TOKEN"

# logout
sbell user logout "$TOKEN"
```

The token is a JWT (`base64(header).base64(payload).sig`). Decoding the payload shows the session identity:

Field	Meaning
`id`	numeric user id (e.g. manage = 637443)
`dmn`	domain - empty for manage, which is normal; permissions are resolved server-side by user id, not from this field
`usr`	username
`exp`	expiry (epoch-like)
`tty` , `hst` , `ipa`	terminal / host / client IP recorded at login

Helpful session commands: `user/whoami` (current identity), `user/list-current-user-sessions` , `user/get-login-history` , `user/logout` . Login is the only call that does not take a token.

How sbell works

`sbell` is the native client to `sballd` 's local UNIX socket. Invocation form:

```
sbell <module> <command> "<params-xml>" ["<token>"]
```

Three things decide whether a call succeeds:

1. **Run as root.** The sballd socket is root-only. As a normal user you get `errno 108` ("Couldn't connect to UNIX socket ... Permission denied"). Use `sudo sbell ...` or a root shell.

2. **Get the params XML shape right.** Each action defines its own root element - it is NOT always `<params>`. `user/login` wants `<params><username>..</username> <password>..</password></params>`; `vm/stop` wants the bare `<vm_key>UUID</vm_key>`; `vm/start` wants `<vm_start><vm_key>..</vm_key> <host>auto</host>...</vm_start>`. When the shape is wrong, sballd often can't extract the key and returns a *misleading* `errno 40 Permission denied` - not a real authorization problem. `/usr/bin/easyshell` is the canonical source of the exact shapes.
3. **Attach a valid token** (all calls except `user/login`).

Every response is the same XML envelope:

```
<response>
  <status>success|failed</status>
  <action_id>UUID</action_id>
  <description>...payload, or on failure <mid>module</mid><errno>N</errno>
<what>message</what>...</description>
</response>
```

Common error numbers:

errno	`what`	Meaning / fix
39	UnAuthorized	No valid session - log in / pass the token
40	Permission denied	Real perm issue, OR (common) a malformed params body so no key was parsed
106	Bad module name	Wrong `<module>`
107	Bad command name	Wrong `<command>` for that module
108	UNIX socket ... Permission denied	Not running as root
137	Username/Password is not correct	Bad login credentials
146	VM '<name>' is running in cluster	VM-state error (e.g. start on a running VM) - note this is from `mid=vm`, not actrepo

The wire protocol underneath (used by both the socket and TCP `:7090`) is a 15-digit zero-padded length prefix followed by the XML - useful if you want to speak to sballd directly instead of shelling out to `sbell`.

Command catalog

The Plaivid front-end drives 97 commands across 30 modules. Each is callable as `sbell <module> <command>` and as `POST /sabad-plaivid/<module>/<command>`. The full set:

Module	Commands
`user`	login, logout, whoami, objects, change-password, download-profile, update-profile, upload-profile, get-login-history, list-current-user-sessions, ldap-sync-users, ldap-users
`vm`	start, stop, kill, reset, pause, resume, screenshot, query-net-quota
`object`	add, delete, section-modify
`sball`	show, master, get-cpu-info, node-power-on, node-power-off
`module`	list, object-type-list
`monitoring`	metrics, system
`network`	nic-list, list-active-ntp
`vnet`	set-traffic-quota
`flash`	create, connect, disconnect, unmount, get-access, grant-access, change-owner
`storage`	file-upload, scan, scan-backup, usbfs-lsblk, usbfs-mount, usbfs-unmount
`usb`	ls-host-usb
`container`	create-instance, import-template, reset-quota
`desktop`	create-version, remove-version, rebuild, change-group, delete-group, remove-template, sync-group-setting
`broker`	run, stop, block, unblock, console, fetch-active-session
`job`	task-run, task-cancel, get-object-actions
`vmbackup`	delete
`vmclone`	run

Module	Commands
`vmmigrate`	run
`vmstrgmigrate`	run
`vmreplicate`	get-slave-list
`fylad`	create, convert, cleaning, resize-flash, manage-access, query-access-report, query-link, remove-link
`fyladbackup`	delete
`fyladkiosk`	check-new-version, update-new-version-info
`inad`	assign-net-policy, query-net-policy, query-url, query-access-report, delete-unused-flashes
`license`	show, system-key, update
`log`	select
`report`	test-email
`talk`	run-command
`change`	password
`help`	help

The ones you'll use most for VM control (params shape in parentheses):

- `vm/stop` (`<vm_key>UUID</vm_key>`) - graceful ACPI shutdown.
- `vm/kill` (`<vm_key>UUID</vm_key>`) - forced power-off.
- `vm/start` (`<vm_start><vm_key>UUID</vm_key><host>auto</host><version_start><version_enable>no</version_enable><version_vkey></version_vkey></version_start><exit_from_replicate>yes</exit_from_replicate></vm_start>`).
- `vm/reset` , `vm/pause` , `vm/resume` (`<vm_key>UUID</vm_key>`).
- `vm/screenshot` (`<params><vm_key>UUID</vm_key></params>`) - note this one DOES use the `<params>` wrapper.

Generic object CRUD: there is no per-type "create vm" command - new VMs, disks, bridges, etc. are created with `object/add` carrying an `<object_section><pvm_..._obj>...</pvm_..._obj></object_section>` body, and removed with `object/delete` . `object/section-modify` edits one section of an existing object.

Introspection (great for a TUI): `module/list` and `module/object-type-list` enumerate modules and object types; `job/get-object-actions` returns the actions currently valid for a given object; `user/objects` runs an XPath-style query (`<params><query>/objects/object/pvm_vmm_virtual_machine</query></params>`) over the objects your session can see.

Host CLI tools & layout

Besides `sbell`, the host ships ready-made wrappers:

Tool	What it does
`pvm-running-vms`	Lists running VMs: ROW, PID, UPTIME, PORT, VCPU, MEMORY, UUID, NAME. Read-only, no login needed. The UUID column is the `vm_key`.
`/usr/bin/easyshell`	Friendly wrapper over `sbell`: `vm_list`, `disk_list`, `iso_list`, `bridge_list`, `vm_start <key> <host>`, `vm_migrate <key> <host>`, `vm_rollback <key> <snap>`, `vm_kill <key>`, `admin_list`, `user_list`. It calls `sbell` without sudo, so run it as root. It is also the clearest reference for each action's params XML.
`/usr/bin/execute_job`	Logs in as the `pvm` system user and runs a scheduler task (`sbell job task-run`).
`sabad`	The Avid Cloud Web API daemon - HTTP on `127.0.0.1:9999`, exposed through nginx at `/sabad-plaivid/` (and `/sabad-inad`, `/sabad-fylad`, `/sabad-rakaard`).
`pvm` meta-service	`systemctl {start,stop,restart} pvm` manages the whole stack (sballd, plogger, pvm-engine, sabad, nginx, ...). Manage this, not the individual services.

Key paths:

Path	Contents
`/etc/pvm/engine/*/manager.conf`	per-module engine config (db, vm, rpc, sbell, ...)
`/etc/pvm/engine/module/manager.conf`	the engine `module_list`

Path	Contents
`/opt/pvm/nginx/conf.d/`	nginx: `sabad.conf` (API routing), `default.conf`, `console.conf`, `admin.conf`, `ssl.conf`
`/var/www/html/plaivid/`	the Flutter web UI (`main.dart.js`)
`/var/lib/pvm/*.xml`	the sballd object store (VMs, users, networks, ...)
`/var/run/pvm/<uuid>.pvmon`	each running VM's QMP control socket (root-only); `system_powerdown` here is the low-level graceful stop
`/var/log/pvm/messages`	sballd action log - every command start/finish with user, client IP and result
`:7090` sballd / `:9999` sabad / `:8181` pvm-engine	listening services

sbell vs the sabad HTTP API

Same commands, two transports. `sbell` talks to sballd's local socket (root, on-host). The HTTP API is what the browser uses - go through it for remote/scripted access.

HTTP request format (this is where it's easy to go wrong):

- `POST https://<host>/sabad-plaivid/<module>/<command>`
- `Content-Type: multipart/form-data` with **three** fields: `version=1.0.0`, `query=` (empty), and `params=<action-xml>`.
- Header `Product: Plaivid`; the session `token` as a **cookie** (set by `user/login`).
- `params` is the **bare action element**, NOT wrapped in `<params>`. Wrong content-type or an extra `<params>` wrapper makes sabad parse out no key, and actrepo returns a misleading `errno 40 Permission denied` - which looks like an auth wall but isn't.
- Login is the exception: it works url-encoded with the `<params>` wrapper.

Worked example - curl (stop then start Hadi-New):

```
base=https://127.0.0.1/sabad-plaivid
VM=481e458c-c6ba-4fe7-88f6-26a59e416568

# 1) login (url-encoded, <params> wrapper) -> token cookie
curl -sk -c cj.txt -X POST "$base/user/login" \
  --data 'params=<params><username>manage</username><password>PASS</password>
</params>' \
  -H 'Product: Plaivid'

# 2) stop (multipart; bare <vm_key>; use --form-string so < is not read as a
file)
curl -sk -b cj.txt -X POST "$base/vm/stop" \
  --form-string 'version=1.0.0' --form-string 'query=' \
  --form-string "params=<vm_key>$VM</vm_key>" -H 'Product: Plaivid'

# 3) start (multipart; full <vm_start> element)
curl -sk -b cj.txt -X POST "$base/vm/start" \
  --form-string 'version=1.0.0' --form-string 'query=' \
  --form-string "params=<vm_start><vm_key>$VM</vm_key><host>auto</host>
<version_start><version_enable>no</version_enable><version_vkey>
</version_vkey></version_start><exit_from_replicate>yes</exit_from_replicate>
</vm_start>" \
  -H 'Product: Plaivid'
```

Worked example - sbell (same thing, on-host, as root):

```
PASS='...'; VM=481e458c-c6ba-4fe7-88f6-26a59e416568
T=$(sbell user login "<params><username>manage</username>
<password>$PASS</password></params>" \
     | sed -n 's:.*<description>\(.*\)</description>.*:\1:p')
sbell vm stop  "<vm_key>$VM</vm_key>" "$T"
sbell vm start "<vm_start><vm_key>$VM</vm_key><host>auto</host>
<version_start><version_enable>no</version_enable><version_vkey>
</version_vkey></version_start><exit_from_replicate>yes</exit_from_replicate>
</vm_start>" "$T"
sbell user logout "$T"
```

A success body carries `<status>success</status>`; for a VM action it includes a `<cluster_status><status>stopped|running</status>...</cluster_status>`. `manage` is fully authorized for these - earlier Permission-denied results were format bugs, not missing rights.

Building a Plaivid TUI

A TUI is a thin client over the command layer above. The core loop is the same whichever transport you pick:

1. **Log in once**, cache the token in memory for the session.
2. **Resolve names to keys.** The UI needs VM name -> `vm_key` (UUID). Get it from `pvm-running-vms` (running only), `easyshell vm_list` (all), or `user/objects` with query `/objects/object/pvm_vmm_virtual_machine`.
3. **Build the action's params XML** (see the catalog) and call the command.
4. **Parse the XML response**: check `<status>`; on failure read `<errno>` / `<what>`; on success read the payload (e.g. `<cluster_status>`).
5. **Refresh** the VM list/state after each action.

Transport - pick one:

Transport	When	Notes
Shell out to `sbell`	On-host TUI, running as root	Simplest. No TLS. Build the XML, read stdout.
sballd wire protocol	On-host, no `sbell` dependency	Connect UNIX socket or TCP `:7090`; frame = 15-digit zero-padded length + XML.
sabad HTTP API	Remote / non-root	`multipart/form-data` to `/sabad-plaivid/...`, token cookie, `Product` header (see previous section).

Suggested first command surface: list VMs (running + defined), per-VM start / stop / kill / reset / pause / resume, live state, and open console/screenshot. Drive menus dynamically with `job/get-object-actions` so each VM only offers the actions currently valid for its state.

Library options:

- **Go** - Bubble Tea / Lip Gloss (great for a polished TUI); shell out to `sbell` or implement the 7090 framing.
- **Python** - Textual or Rich; `subprocess` to `sbell`, `xml.etree` to parse. Fastest to prototype.
- **Rust** - ratatui + a small socket client.

Watch out for: always run the sbell/socket path as root; keep each action's params shape exact (the `<params>` wrapper trap); treat `errno 40` on a well-formed, authorized call as a likely params-shape bug; and remember `vm_key` is the UUID from `pvm-running-vms`, not any display name.